

CCAR-F — Initial Assessment Results

Claude Certified Architect – Foundations · Learning path: Exam-Weighted · Date: 2026-08-27 · 15 questions (3 per domain)

Overall: 12 / 15 correct — 80% (PASS — threshold 72%)

Domain	Topic	Score	Accuracy
D1	Agentic Architecture & Orchestration	2 / 3	67%
D2	Tool Design & MCP	3 / 3	100%
D3	Claude Code Configuration & Workflows	2 / 3	67%
D4	Prompt Engineering & Structured Output	3 / 3	100%
D5	Context Management & Reliability	2 / 3	67%

Domain 1 — Agentic Architecture & Orchestration (2/3)

Q1 EASY Your answer: A · Incorrect

You are building an agentic loop in Python using the Anthropic SDK. After calling `client.messages.create(...)`, you receive a response object and need to decide whether to keep looping or stop.

Which field on the API response object should your loop inspect to determine whether the model has finished or is requesting a tool call?

- A `response.content[0].type` YOUR ANSWER
- B `response.stop_reason` CORRECT
- C `response.usage.output_tokens`
- D `response.model`

Explanation: `stop_reason` signals why the model stopped. `"tool_use"` means it is requesting tool calls (continue looping); `"end_turn"` means it is finished (exit). `content[0].type` only tells you the type of the first content block — not the canonical termination signal.

Q2**MEDIUM****Your answer: A · Correct**

An engineer implements an agentic loop that executes tool calls and feeds results back. During a long-running task the model returns `stop_reason: "tool_use"` with two `tool_use` content blocks in a single response.

What is the correct way to return the results of both tool calls to the model in the next API request?

A Append the model's full assistant message to the conversation, then add a single user message containing both `tool_result` content blocks. **CORRECT · YOUR ANSWER**

B Make two separate API calls, each with one `tool_result` block.

C Replace the last user message with a combined message containing both `tool_result` blocks, discarding the assistant turn.

D Add each `tool_result` as a separate user turn so the model sees alternating user/assistant messages.

Explanation: All results for multiple tool calls in one turn go back together in a single user message, one `tool_result` per `tool_use_id`. The triggering assistant turn must remain in history to preserve alternating structure.

Q3**HARD****Your answer: C · Correct**

Auditing an agentic loop processing financial transactions. It occasionally cycles 30+ iterations before terminating, consuming thousands of tokens on tasks that should take 3–5 iterations. Logs show `stop_reason: "tool_use"` on every iteration of the runaway sessions.

Which combination of root causes and mitigations best addresses this runaway loop problem?

A Model ignoring `end_turn` signals → switch to a newer model.

B Loop not including assistant message in history → always include the previous assistant turn.

C Tool results likely return ambiguous/error states the model loops on without converging → add structured errors with `is_error:true`, an iteration cap, and observability to log each tool result. **CORRECT · YOUR ANSWER**

D Parallel execution race conditions → switch all tool calls to sequential.

Explanation: Persistent `"tool_use"` indicates a resolution cycle on ambiguous/error results. Layered fix: structured errors (`is_error:true`) so the model can reason about failure, a hard iteration cap as a safety backstop, and observability to diagnose the pattern.

Domain 2 — Tool Design & MCP Integration (3/3)

Q4 EASY Your answer: A · Correct

A developer registers a tool called `get_user` with no description field. Claude sometimes calls it when it should call `get_account`, and vice versa.

What is the **PRIMARY** reason Claude confuses these two tools?

A Tool descriptions are the primary signal Claude uses to decide which tool to call; without them, Claude can only guess from the name alone. **CORRECT · YOUR ANSWER**

B The tool names are too short and should be more verbose.

C Claude always calls the first tool listed when unsure.

D The API requires a minimum of three tools before disambiguation activates.

Explanation: Descriptions are the primary tool-selection mechanism. With no description, only the name remains as signal — insufficient to distinguish semantically similar tools.

Q5 MEDIUM Your answer: A · Correct

A team has `search_knowledge_base` and `search_web`. Claude calls `search_web` for answers already in the internal knowledge base, increasing cost and latency.

Which description strategy best fixes the tool selection problem?

A Add explicit disambiguation: `search_knowledge_base` covers internal/proprietary/company-specific info and is tried first; `search_web` is for public info not in the knowledge base. **CORRECT · YOUR ANSWER**

B Remove `search_web` entirely.

C Merge into one search tool with a source parameter.

D Prefix tool names with priority numbers.

Explanation: Add domain + preference-order disambiguation language to the descriptions. Don't delete capability, collapse to a mode/source param (anti-pattern), or hack ordering into names.

Q6 **HARD** **Your answer: C · Correct**

A financial assistant has `get_account_balance`, `get_transaction_history` (settled), and `get_pending_transactions` (unsettled). Claude calls `get_transaction_history` for pending charges, which returns no pending items.

Which combination of description changes most precisely solves this three-way disambiguation problem?

- A Add "most commonly used tool" to `get_transaction_history`.
- B Add a routing system prompt keyed on the word "pending".
- C Add cross-references between all three: each description names its sister tools and their boundaries (history does NOT include pending; pending will NOT appear in history; balance reflects settled only). **CORRECT · YOUR ANSWER**
- D Merge history and pending into one tool with a status filter.

Explanation: Cross-referencing descriptions creates a web of mutual disambiguation — each tool actively repels queries meant for another. More robust than keyword routing, artificial priority, or merging.

Domain 3 — Claude Code Configuration & Workflows (2/3)

Q7 **EASY** **Your answer: D · Correct**

A developer wants personal preferences (coding style, default language) that apply to every project on their machine.

Which file path is the correct location for user-level CLAUDE.md instructions that apply globally?

- A `./CLAUDE.md` in the root of each project
- B `.claude/rules/CLAUDE.md` in the project directory
- C `~/CLAUDE.md` in the home directory root
- D `~/.claude/CLAUDE.md` **CORRECT · YOUR ANSWER**

Explanation: User-level memory lives at `~/.claude/CLAUDE.md` — note it must be inside the `.claude/` directory (distinguishing it from the C trap, `~/CLAUDE.md`).

Q8 **MEDIUM** **Your answer: B · Incorrect**

Three `CLAUDE.md` files: `~/.claude/CLAUDE.md`, project root `./CLAUDE.md`, and `./frontend/CLAUDE.md`. Claude Code is operating inside `frontend/`. All three contain a conflicting rule about TypeScript strict mode.

Which rule takes precedence when working inside the `frontend/` directory?

- A User-level rule — personal preferences override project settings.
- B Project-level rule — it is version-controlled and carries team authority. **YOUR ANSWER**
- C Directory-level rule in `./frontend/CLAUDE.md` — the most specific (closest) scope wins. **CORRECT**
- D All three merged in load order; no single rule wins.

Explanation: Resolution order is user → project → directory, most specific wins. Project overrides user but is itself overridden by a directory-level file. Version-control/team-authority is NOT the tie-breaker — scope specificity is.

Q9 **HARD** Your answer: C · Correct

~/.claude/CLAUDE.md says 2-space; ~/.CLAUDE.md says 4-space; ./backend/CLAUDE.md does @import ~/.claude/rules/style.md and that style.md says "use tabs". Editing ./backend/services/auth.ts.

What indentation rule applies, and why?

A 2-space (user-level) — imports cannot override user baseline.

B 4-space (project) — version-controlled, beats imported files.

C Tabs from style.md — the directory-level ./backend/CLAUDE.md is the most specific scope and its @import content is inlined into it, winning over higher scopes. **CORRECT · YOUR ANSWER**

D 2-space — user-level used as @import tiebreaker.

Explanation: @import content is inlined into the importing file and inherits its scope. The directory-level file is most specific, so its imported "use tabs" wins.

Domain 4 — Prompt Engineering & Structured Output (3/3)

Q10 **EASY** Your answer: B · Correct

A prompt says "flag uncertain items." A colleague suggests "flag items where your confidence is below 0.8."

Why is the revised instruction superior for production use?

A It is shorter and easier to parse.

B It replaces a subjective judgment call with an objective, measurable threshold, reducing inconsistency across runs. **CORRECT · YOUR ANSWER**

C Claude tracks numeric confidence scores internally, triggering a built-in feature.

D Qualitative guidance is always wrong.

Explanation: An objective threshold makes the criterion reproducible across runs and models. (C is a trap — Claude exposes no real internal confidence score; the number works as a clearer instruction.)

Q11 **MEDIUM** Your answer: B · Correct

A company extracts entities from regulatory filings with the prompt "Extract all important entities." Claude misses jurisdiction codes and monetary amounts inconsistently.

Which prompt revision will **MOST** reliably improve extraction completeness and consistency?

A Add "Be thorough".

B Replace with an explicit enumeration: (1) company names, (2) jurisdiction codes (e.g., US-NY, EU-DE), (3) monetary amounts with currency symbols. **CORRECT · YOUR ANSWER**

C Ask Claude to read the filing twice.

D Switch to a larger model.

Explanation: Explicit enumeration with examples defines what "important" means so Claude applies the same criteria every time. The gap is definitional, not model capacity or effort.

Q12**HARD****Your answer: C · Correct**

A medical triage system classifies symptoms into 3 categories; 22% of borderline cases miscategorized. Remedy 1: add 10 few-shot edge-case examples. Remedy 2: replace qualitative descriptions with explicit rule-based criteria.

In a high-stakes medical triage application, which approach is more appropriate and why?

A Remedy 1 only — examples beat rules in ambiguous medical domains.

B Remedy 2 only — rules are more auditable; examples risk wrong edge cases.

C Both combined — explicit rules define the decision boundary while examples anchor borderline interpretation; both essential for safety-critical classification. **CORRECT · YOUR ANSWER**

D Neither — Claude should not be used for medical triage.

Explanation: Rules define the decision space (auditable, controllable); examples demonstrate application at the boundaries. Rules-only leaves interpretation gaps; examples-only risks overfitting. Together = complete rubric.

Domain 5 — Context Management & Reliability (2/3)

Q13**EASY****Your answer: D · Incorrect**

A developer notices important user preferences mentioned at the very beginning of a long conversation are ignored later in the session.

What is the PRIMARY architectural explanation for why Claude loses track of information in the middle of a long context window?

A The tokenizer strips repeated content after the first occurrence.

B Claude applies equal attention to every token, so nothing is lost.

C The "lost in the middle" phenomenon — models attend less reliably to content far from the start or end of the context. **CORRECT**

D Claude automatically deletes content older than 10 turns to stay within the window. **YOUR ANSWER**

Explanation: "Lost in the middle" — start (priming) and end (recency) get more attention; the middle is underweighted though still present. Claude has NO automatic deletion; this is an attention problem, not a storage one.

Q14

MEDIUM

Your answer: A · Correct

A multi-session research assistant produces a progressive summary after each session. In session three it has forgotten a key constraint from session one that was only briefly mentioned in session one's summary.

Which limitation of progressive summarization most likely caused this information loss?

A Progressive summaries compress content each round, and details not reinforced in later sessions gradually drop out.

CORRECT · YOUR ANSWER

B Temperature 0 prevents creative retention of edge cases.

C Summarization always prioritizes the most recent turn.

D Session boundaries automatically reset all prior constraints.

Explanation: Each compression round drops detail; briefly-mentioned constraints/edge cases are disproportionately likely to vanish. Fix: pin or re-inject critical constraints rather than trusting progressive compression.

Q15

HARD

Your answer: B · Correct

A compliance assistant's system prompt has regulatory rules (3k tokens at position 0), a running decision log (grows to 40k tokens in the middle), and a checklist (2k tokens at the end). After 150 turns, Claude inconsistently applies the rules but reliably follows the checklist.

What does this reveal about attention distribution, and what architectural change fixes the rule inconsistency?

A Anomalous — a model defect requiring vendor escalation.

B The 40k-token decision log pushes the rules into the effective middle despite their nominal position 0, reducing salience; fix is to re-pin the rules at the END of context after the log.

CORRECT · YOUR ANSWER

C Position 0 is the lowest-attention zone; move rules to the middle.

D The checklist is short so Claude prefers it; shorten the rules to match.

Explanation: "Position 0" is only nominal — the large decision log after it buries the rules in the effective middle, while the checklist at the true tail stays in the high-attention recency zone. Fix: re-anchor the rules near the end (append a concise summary as the last block or per-turn).